1 **Design & Implementation of Human-Computer Interfaces**
NPTEL-MOOCS

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

2 **L19 – DFD+ERD**

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

3

4 **Recap**

- We learned basics of system design
- Today we shall learn about one "language" for expressing system (code) design
  - DFD

5 **Background**

6 **Design Concerns**

- Two issues
  - Where to start?
  - How to represent (design language)?

7 **Where to Start?**

- SRS – Software Specification Language
- Two phases
  - Preliminary (high-level) design
  - Detailed design (also called module-specification document)

8 **High-Level Design**

- Identification of modules
- Control relationships between modules
- Definition of interfaces between modules

9 **Basic Design Approaches**

- Function oriented – basic abstractions are functions

10 **Basic Design Approaches**

- Object oriented – basic abstractions are objects (instantiation of class)

11 **How to Represent?**

- With a "language"
- DFD (Data Flow Diagram) is one such language to "express" design

- Used for functional design approach

12 **DFD - Basics**

13 **DFD**

- Represents "flow of data" through a process or a system
  - Focus on data *"movement"* between external entities and processes, and between processes and data stores

14 **Why DFD?**

- Provides overview of
  - What data processed by a system
  - Transformations that are performed
  - Which data are stored
  - What results are produced and where they go
- Graphical nature makes it a good communication tool between
  - User and (system) designer
  - Designer and developer

15 **Components of DFD**

- Source/Sinks (external entity)
- Processes
- Data stores
- Data flows

16 **Component Representation**

17 **DFD Naming Guidelines**

- Process ☐ verb phrase
- Data flow label ☐ noun: name of data
- Data store ☐ noun
- External entity ☐ noun

18 **External Entity**

- People or organizations that send data into or receive data from system
- They either supply or receive data
  - Source – supplies data to system
  - Sink – receives data from system

19 **Process**

- Process – series of actions that transform data
- Notation
  - Straight line with incoming arrows ☐ input data flows

- Straight lines with outgoing arrows ☐ output data flows
- Labels assigned to data flow

20 **Data Store**

- Represents permanent data used by system
  - Data can be written into data store – denoted by an incoming arrow
  - Data can be read from a data store – denoted by an outgoing arrow

21 **Data Flow**

- Data flow - depicts actual flow of data between elements
  - Connects processes, external entities and data stores

22 **Data Flow**

- Notation – an arrow with label
  - Generally unidirectional
  - If same data flows in both directions, double-headed arrow can be used

23 **Data Flow Rules**

24 **3 Incorrect Flow**

25 **Hierarchy**

26 **Representing Complex Designs**

- Real-life systems can be very big
  - Involving large number of processes and data stores
- Representing such systems with DFD difficult
  - Very complex diagram
  - Difficult to understand
  - Difficult to modify

27 **Representing Complex Designs**

- Way out – decomposition
  - Create hierarchy of "levels"

28 **DFD Levels**

- At least 3
  - Level 0 – context diagram
  - Level 1 – overview diagram
  - Level 2 – detailed diagram
- There can be further levels, if required

29 **Level 0 (Context Diagram)**

- Contains only one process

- Represents entire system
- Data arrows show input and output (from/to external entities)
- Data stores NOT shown - implicitly contained within system

30

31 **Level 1 (Overview Diagram)**

- Utilizes all four elements
  - Data stores first shown at this level

32

33 **Level 2 (Detailed Diagram)**

- Detailed representation of level 1 processes
- Like level 1, utilizes all four elements

34

35

36

37

38 **Creating Good DFD!**

- Use meaningful names for data flows, processes and data stores.
- Use top down development starting from context diagram and successively levelling DFD
- Stop decomposition if levels become trivial (no significant change)
- Remember - only previously stored data can be read
- Remember - data stores cannot create new data